PA-consistency machines: comparison and refined reduction search

Research report

November 13, 2026

Abstract

The requested construction has at most 195 working states, a single binary tape initially blank, and a PA-provable equivalence between nonhalting and consistency of ordinary first-order PA. **No verified machine meeting the state-count target was obtained.** The smallest accepted state count in this package remains 196.

We compare all nine designated 208-state tables in the incoming archives, the specified 209-state predecessor, and the published 196-state table. The nine 208-state presentations represent two rooted transition-graph isomorphism classes; their final deletion is already absent from PA196. An earlier report in the same incoming directory had reached this comparison, so the new work focuses on stronger physical invariants, joint control rewrites, and logical simplifications beyond that report.

New independently checked certificates establish a zero/nonzero invariant and a system of difference bounds for the 3,812-site source program. Their intersection reduces 4,959 generic instruction cases to 4,954 reachable cases. A generalized executor permits repeated register excursions, extra spare registers, and increased tape gaps while preserving the nine logical registers. The resulting broader searches still do not produce a certified sub-196 machine. We also give an all-instance countervaluation refuting a tempting replacement of the first quantifier head, and explain a completeness obligation concealed by a proposed removal of freshness tests.

Contents

1 Result, conventions, and frozen sources **2**

2 Comparison of the designs **2**

3 The logical calibration and its formal status **4**

4 A stronger physical verification interface **5**

5 Inductive source certificates **6**

6 Logical simplifications that require care **7**

7 New search families and evidence limits **8**

8 Reproduction and remaining work **9**

1 Result, conventions, and frozen sources

The intended statement is

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M) \longleftrightarrow \mathrm{Con}(\mathrm{PA}), \qquad \#\,\mathrm{workingStates}(M) < 196. \tag{1}$$

Here PA is ordinary first-order Peano arithmetic with induction for all formulas. A working state has one transition on each of two symbols. The halt label has no transition and is not counted. The tape is unbounded in both directions and initially zero everywhere. These are the conventions of the supplied machines; enlarging the alphabet or supplying a nonblank initial program would change the comparison.

The package does not contain an accepted 195-state table, nor does it claim an impossibility theorem. A rejected candidate, a lower grammar score, or a solver bound for a restricted family is not a construction satisfying (1).

Source	Frozen revision
ProveIt, current inspection	`f1f076e31995f10460838d1846456f4bb9c8ae80`
BusyBeaver, PA196	`407cdc356f6b586635e71ac92be394255f4bfea4`
BusyBeaver, requested PA209	`69d98241a721ead20b4f0b17df9062950b40fbfe`

Table 1: Commit identities used for this investigation. Input hashes and the current archive census accompany the report.

The current incoming directory contains twelve ZIP archives. One is an earlier response to this comparison, `pa_tm_research_2026-11-13.zip`. It contains extensive negative searches and an alternative 196-state layout [5]. Its results were inspected before selecting new search families. Its comparison inputs are unchanged in the current snapshot. The accompanying comparison code is retained with attribution, and the current-input audit supplies a separate provenance check.

2 Comparison of the designs

2.1 Nine presentations of two 208-state tables

The five construction packages below contain the nine designated final 208-state inputs. The other incoming archives contain comparisons, calibration, or attempted reductions, rather than additional accepted 208-state designs.

Construction package	Class A	Class B
`U208_ConPA_2026-10-29`	1	0
`U208_ConPA_Conditional_Context_2026-10-29`	1	1
`U208_ConPA_Entry_Context_2026-10-29`	1	1
`U208_ConPA_History_2026-10-29`	1	1
`u208_research_2026-10-29`	1	1
Total	5	4

Table 2: Exact rooted, action-preserving isomorphism classes.

The comparison canonicalizes a deterministic rooted transition graph by breadth-first discovery, reading the zero edge before the one edge. It retains both output symbols, both directions, the start

state, and the distinguished halt label. Equality of these canonical serializations supplies an explicit state bijection. The reproduction program verifies the bijections, rather than relying only on digest equality.

The two classes have different tree associations and control layouts, but the same resolved 3,909-occurrence register program. Their maximum source-path depths are 42 and 41. The common last reduction identifies native roles 172 and 211 by rereading a bit of the incoming path. These are two implementations of the same saving, not nine independent deletions.

2.2 What changes between PA209 and PA196

Design	States	Rows	Source sites	Generic cases
U208 class A	208	416	3,909	5,084
U208 class B	208	416	3,909	5,084
Specified PA209	209	418	3,948	5,135
Published PA196	196	392	3,812	4,959

Table 3: A decrement contributes two cases, one for each outcome. Source size and physical state count are different measurements.

The 209-state table specified in the question is not identical to the U209 parents included in the U208 packets. Native-label maps in the Lean sources allow a meaningful comparison despite different literal labels. The specified PA209 and PA196 have 134 native working-state labels in common. Of the corresponding 268 rows, 265 agree exactly. The differences on that intersection are

native row	PA209	PA196
$(164, 0)$	$0\ L\ 311$	$0\ R\ 77$
$(190, 1)$	$1\ L\ 308$	$1\ R\ 99$
$(101, 1)$	$1\ R\ 115$	$1\ R\ 1367.$

Overall PA196 deletes 75 old states and introduces 62, giving $209 - 75 + 62 = 196$. Native roles 172 and 211 are already absent. Consequently the U208 final saving has no pair of remaining roles to act on.

The 196 states consist of 36 engine states, 13 leaf routers, and 147 reader states implementing 159 distinct internal tree nodes. Its axiom body has 201 macro items and 68 distinct internal nodes; six compatible history-based identifications implement that body with 62 reader states. Reassociating the word changes both subtree sharing and available identifications, which is why fewer tokens need not mean fewer states.

2.3 Logical changes behind the thirteen-state saving

Let $p(a, b) = \binom{a+b+1}{2} + b$. PA196 uses

$$\mathsf{I}(a, b) = 4p(a, b), \quad \mathsf{M}(a, b) = 4p(a, b) + 1, \quad \mathsf{U}(a, B) = 4p(B, a) + 2, \quad \mathsf{E}(a, b) = 4p(a, b) + 3.$$

Truth has code 0 and the intended contradiction is code $1 = \mathsf{M}(0, 0)$. Relative negation is $\mathsf{N}(B) = \mathsf{I}(B, 1)$. Compared with PA209, the universal constructor exchanges its pairing arguments. A primitive equality head is removed; equality vacuity is added under the existing separation guard. The remaining heads are rearranged to improve their shared compilation [1, 2]. These are changes to the proof supplier, not merely state renamings.

The literal PA196 table retained in this package has SHA-256

`e8bbccd4cfa325251ca95066db116b5aa766e404b0971c33c02772be1c3617a8`.

The start is literal state 0, the halt label is 196, and the only row entering the halt label is $(63, 1, 1, R, 196)$.

3 The logical calibration and its formal status

The explicit published arithmetic endpoint is

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M_{196}) \to \mathrm{Con}(\mathrm{PA}), \tag{2}$$

named `PA196.Arithmetic.pa_proves_nonhalting_consistency`. The repository also proves external blank-tape nonhalting and two-sided PA independence of its exact halting sentence [4]. Those endpoints do not themselves assert the reverse internal implication in (1). In particular, using a standard model in Lean's metatheory is not an internal PA proof.

The earlier incoming report supplies a written syntactic route to the converse, which was independently audited in this investigation. The argument translates an accepted command derivation into a PA refutation. It does not appeal to a uniform truth predicate for arbitrary formulas. The following explains the substance and the remaining formal boundary.

3.1 Uniform proof production under the binary interpretation

Interpret $\mathsf{M}(a, b)$ as the statement that bit x_b of x_a is one, $\mathsf{E}(a, b)$ as equality of x_a, x_b, implication as implication, and $\mathsf{U}(a, B)$ as $\forall x_a B$. PA proves that a natural number does not contain itself as a binary digit position: a nonzero bit at x would give $2^x \leq x$, whereas PA proves $x < 2^x$. Thus the interpreted code 1 has a uniform PA refutation.

The ordinary heads have uniformly constructible PA derivations. The only substantial supplier is the pointed-separation gate. For a fixed arithmetical formula $\varphi(u)$, induction in PA proves finite comprehension below any bound: there is a number whose bits below the bound select exactly the u satisfying $\varphi(u)$, with other bits zero. The proof is obtained by induction on the bound, adding 2^u precisely in the $\varphi(u)$ case. This uses one induction instance containing φ; it does not require PA to evaluate arbitrary formula codes.

For distinct atom labels, apply this construction to the elements of the given finite set together with the distinguished point. For coincident labels, membership of a number in itself is false, and a singleton or the empty set gives the required pointed instance. The guard ensures that the witness label is distinct from both atom labels. The numerical code bound ensures that it is also fresh for the property formula. Equality and membership vacuity then have direct PA proofs.

Annotate every theorem-valued slot in a finite command execution by the code of its interpreted PA derivation. Axiom supply inserts the uniform proof templates; modus ponens and generalization transform proof codes; stack operations transport annotations. A halt exposes code 1 and therefore yields a PA refutation. All these operations can be implemented by primitive recursion on finite proof and execution codes.

The required internal theorem is the syntactic statement that this compiler sends every accepted finite trace to a PA refutation. Its contrapositive is $\mathrm{Con}(\mathrm{PA}) \to \mathrm{Nonhalt}(M_{196})$. Once the uniform correctness and literal-trace binding obligations are discharged, together with (2) it gives the desired mathematical calibration for the 196-state construction.

3.2 What is and is not claimed here

The proof-producing construction is a sound ordinary mathematical route. An end-to-end formal endpoint still needs its exact proof-code functions, their correctness proofs, and the binding to the literal arithmetic trace predicate. This package does not claim a new Lean proof of that internal biconditional. Nor has the complete upstream Foundation/Mathlib development been rebuilt during this investigation. The unchanged pinned upstream proof is a source; the new Python and C++ computations below are separately described executable evidence.

4 A stronger physical verification interface

The new searches keep the logical calculus unchanged and relax a convenience of the existing checker: one source instruction need not perform exactly one register-file excursion. It may make additional excursions whose net logical effect is the same.

4.1 Projection to the nine logical registers

The physical file encodes a value v by $0\,1^{v+1}$. The logical-to-physical register permutation is

$$\pi = (6, 2, 0, 4, 3, 7, 8, 5, 1).$$

A valid file has at least ten registers. Only physical indices 0 through 8 are used by the source program; later registers are spare storage. An engine increment appends a spare register of value one, and a positive decrement shifts the start of the file right by one cell.

Write $F(i, g, v)$ for the complete canonical frame with source site i, gap $g \geq 48$, and physical file v. Let $\mathrm{proj}(v)$ denote its nine logical registers. Equality of this projection is weaker than equality of the whole file, but it is the equality relevant to the source computation.

Theorem 4.1 (Sufficient physical contract)**.** *Suppose a binary table T has the following properties. Its blank startup reaches a canonical frame representing the source seed. An invariant $J(i, u)$ holds for that seed and is preserved by every source instruction. For each nonhalting source instruction and every valid frame satisfying J, the table reaches in a positive number of steps a canonical frame $F(i', g', v')$, with $g' \geq 48$, whose projection is exactly the source successor. A source halt reaches the literal halt label, and every finite nonterminal simulation segment has defined transitions. Then blank-tape halting of T is equivalent to halting of the source computation.*

Proof. Iterate the contracts along a finite source run. The invariant supplies each next precondition, while the lower gap and file-length bounds permit repeated use of the contracts. A halting source run yields a finite physical halting trace.

Conversely, if the source does not halt, the same induction gives an infinite chain of canonical frames joined by nonempty finite physical runs. For every natural n, concatenating the first n segments gives at least n physical steps with defined transitions. A finite physical halting run would contradict this fact, by determinism. The positive-length condition excludes an infinite source computation represented by only finitely many physical steps. □

The argument uses only finite trace concatenation, determinism, and induction. When the invariant and contract witnesses are arithmetized with PA-provable total constructions, the halting equivalence is formalizable in PA. It does not by itself supply the logical calibration of the source theory.

4.2 The generalized symbolic box

The symbolic executor keeps a vector of offsets $\delta \in \mathbb{Z}^9$, a marker location, and a count of appended spares. The current register values are the initial values plus δ. Each use of the unchanged engine's mark, increment, or decrement lemma is instantiated on the current complete file.

For a current expression $v_r + \delta_r$, a zero branch is allowed only when its upper bound is zero; a positive branch is allowed only when its lower bound is at least one. An earlier positive value does not justify a second positive decrement after subtracting one. The implementation rejects an ambiguous extra test unless it has an appropriate further case certificate.

The final comparison requires the exact successor site and nine-register effect, an unmarked well-formed file, a restored canonical path, and at least the required gap. Extra appended spares are allowed. These are the existential freedoms in Theorem 4.1, not permission to ignore arbitrary tape corruption. All engine rows used by its register lemmas must remain unchanged. A rewrite of one of those rows cannot be certified by silently retaining the original box oracle. The new Python executor is itself a trusted verification component; the upstream Lean executor-soundness theorem does not automatically certify this modified implementation.

5 Inductive source certificates

5.1 Zero/nonzero masks

For every site i, the first certificate stores a set S_i of nine-bit patterns. Bit r is one precisely when logical register r is positive. Increment sets that bit. A zero decrement follows its zero edge without changing the pattern. A positive decrement follows its other edge and allows either a positive or zero result for that register. Jumps preserve the pattern. The source restart maps its terminal program counter to site zero.

The seed is site 25 with logical register 0 equal to 2 and every other logical register zero. The independent certificate checker verifies that its pattern is admitted and that every abstract successor remains in the appropriate stored set. It checks 274,880 site-pattern pairs and 326,688 successor edges. This closure check is independent of the worklist algorithm that generated the certificate.

5.2 Difference bounds

The second certificate uses matrices with entries

$$D_{rs} \quad \text{meaning} \quad x_r - x_s \le D_{rs},$$

where a tenth coordinate is the constant zero. Increment and decrement translate the relevant row and column. A zero test adds $x_r \le 0$; a positive test adds $x_r \ge 1$. Canonical closure propagates the constraints. Widening is used only to discover a terminating overapproximation. The independently checked final certificate is what matters.

The C++ generator performs 21,336 worklist iterations. An independent Python checker uses unbounded integers and genuine infinity, verifies the exact source instructions, checks the seed, verifies matrix closure, and checks inclusion of every transfer in its successor matrix. There are 4,953 nonempty transfer edges and five empty branch transfers. The bottom sites are 8, 3774, and 3775.

Compared with the mask certificate alone, the difference bounds prove that logical register 6 is zero at sites 2, 5, and 7. In particular the positive branch at site 7 and site 8 can be omitted. Intersecting the two independently inductive invariants remains sound. At a decrement, the branch

Instruction-contract interface	Cases
Original, both outcomes at every decrement	4,959
Zero/nonzero invariant	4,956
Intersection with difference bounds	4,954

Table 4: The smaller case set is justified by inductive certificates, not by a finite sample of the actual proof search.

assumption further refines the matrix before extracting zero, positive, and numerical interval information.

The five omitted generic cases are the positive branch at site 7, site 8, the positive branch at site 3773, and sites 3774 and 3775. All retained baseline contracts pass the generalized symbolic executor. All 392 rows are nevertheless still covered by the retained physical traces, the exact startup, or the protected engine. No whole row becomes disposable merely from these invariants.

5.3 A reduced joint invariant

We also feed common mask facts back into the difference bounds and discard masks incompatible with the resulting bounds, then propagate again. The final joint certificate stabilizes after three refinement rounds and contains 274,768 site-pattern pairs. A separate Python checker verifies its reduction, seed, and every joint transfer directly. It still yields 4,954 instruction cases. Its additional useful precondition is that physical register 2 is zero on the positive branch of physical register 8 at source site 4. Numerical lower and upper bounds, rather than only zero/nonzero flags, are also passed to the generalized executor, allowing it to recognize an exact value one that becomes zero after an extra decrement.

6 Logical simplifications that require care

6.1 A countervaluation for the shorter quantifier head

The first PA196 head is

$$h_1(d,H) = (\exists x_d\, \mathsf{U}(d,H)) \to H, \qquad \exists x_d B := \mathsf{N}(\mathsf{U}(d,\mathsf{N}(B))).$$

Replacing it by $\mathsf{U}(d,H) \to H$ looks attractive because the latter formula is substantially shorter. The following gives a decisive obstruction to recovering the existing calculus from this replacement.

Proposition 6.1. *Keep every other exact PA196 head, the gate, modus ponens, and generalization, but replace h_1 by $\mathsf{U}(d,H) \to H$. The original h_1 and the explosion schema $1 \to H$ are not derivable in that modified numeric calculus.*

Proof. Define a Boolean valuation of numeric formula codes recursively. Code 0 is true; implication is Boolean implication; $\mathsf{U}(d,H)$ has the value of H; and both $\mathsf{M}(a,b)$ and $\mathsf{E}(a,b)$ are true exactly when their numerical labels are equal. The exceptional equality $\mathsf{I}(0,0) = 0$ is respected because true implies true. The component-code bounds make the remaining recursion well-founded.

Code $1 = \mathsf{M}(0,0)$ is true, while code $9 = \mathsf{M}(0,1)$ is false. Every proposed replacement head and quantifier-commutation head is an identity. The two membership-rewriting heads follow from equality of their numerical labels. The existential equality head is true because its outer relative negation has consequent code 1. The same observation makes every unguarded gate true, and a

failed guard returns true code 0. The distribution head becomes $A \to ((A \to H) \to H)$, a Boolean tautology, as is the restricted classical implicational head. Modus ponens preserves truth, and generalization preserves truth since the quantifier is interpreted as the identity.

But $h_1(0, 9)$ has true antecedent and false consequent, and $1 \to 9$ is false. Neither can therefore be derivable. □

This is an all-instance countervaluation of the numeric rules, not a claim that it is a first-order model of arithmetic. It refutes a proposed recovery argument; it does not prove that every possible new PA interpretation in the modified calculus is impossible. The supplied script performs 23,328 supplementary finite checks, but the schema argument above is the proof.

6.2 Why a larger fresh label does not simply remove the guard

Another proposal changes only the gate's parameter after constructing the heads, replacing c by $C = c + H + D + 1$. Then $C > H, D$, and the two equality comparisons can be removed. Soundness is straightforward. Padding a desired property with double negation can make its formula code exceed $H + D$, which recovers pointed separation for that property.

However, the same gate also supplies atomic equality and membership vacuity at its label C. The published completeness proof uses vacuity at every label distinct from the atom labels, including labels smaller than them. Restricting the supplied label to $C > H + D$ removes those instances. Padding a property also changes the bound witness label; it does not recover arbitrary low-label vacuity. One needs a new derivation of those instances, or a refutation compiler with a suitable label-order invariant. Neither is supplied by the padding observation alone.

6.3 The role of ConciseIPRA

The ConciseIPRA material optimizes a proof checker for intuitionistic primitive recursive arithmetic [6]. Its ideas about restricted equality transport, omitted certificate fields, and shared computation are useful here. But its target theory and cost metric differ. Equivalence with iPRA does not provide the needed PA-consistency calibration, and compressed Python size does not give a two-symbol state count.

The PA196 equality reduction already illustrates the relevant principle: retain the equality interfaces required by the interpretation, and derive additional laws from them. Removing quantifier commutation would need a further idea. The existing reverse membership and basic equality interfaces do not use it, but vacuity through quantified formulas and the subsequent bound-variable renaming do. Those are the precise obligations a deletion must replace.

7 New search families and evidence limits

The accompanying receipts record the exact scope and outcome of each completed search. They distinguish literal finite runs, symbolic universal contracts under stated preconditions, discovery heuristics, and independent inductiveness checks.

The fixed-table work includes simultaneous pair aliases in which the retained state may be revisited, shared new discriminators with modified write symbols and both directions, coupled discriminator rows, and mergers tested under the generalized register interface. Unlike an independent one-row substitution, a simultaneous alias must be executed in the modified table: a locally valid excursion may otherwise become a cycle after another state is identified.

The original-table simultaneous search examined 14,706 state pairs and 3,295,730 joint row combinations without a survivor in its local-window model. The generalized original-row-choice quotient search tested 58,490 mutants in its recorded mask-based run, again without a survivor. The final difference-bound, reduced-product, and numerical-interval rerun examined 63,595 admissible mutants, also without a survivor. These are repeated checks of overlapping families; their counts must not be added as though their search spaces were disjoint. In a different family, 297 oriented one-row folds with 784 possible replacement actions gave 232,848 candidate tables. None passed the final 4,954 reachable contracts. A shared-discriminator family of 332 combined candidates was also rechecked with the refined generalized contracts and yielded no accepted table.

We tested classical-equivalent presentations of the remaining heads and gate while retaining the original first head and implicational bootstrap. The 6,912 variants included contraposition of selected implications and different gate associations. The best inherited-dictionary grammar score stayed at 68 internal axiom nodes. This is a result for that parsing model, not a lower bound for other grammars or machines.

Four alternative 196-state tables replace the closing AX break by a decrement of a register known to be zero there. The relevant logical registers are 3, 4, 5, and 6. Their constrained instruction checks pass, and each has the same exact 58,154-step startup endpoint. They give additional search starting points, but no state-count improvement. Their source invariant and verification level are recorded separately; they are not represented as fresh Lean kernel theorems.

More concretely, replace just dense row $(193, 1)$ of the published table, whose action is $0\ L\ 131$, by the following action:

tested logical register	3	4	5	6
new dense action	$1\ R\ 17$	$1\ R\ 113$	$1\ R\ 44$	$1\ R\ 21.$

The source proof already establishes that these registers are zero at site 3692 before the closing instruction. A zero decrement has the same control destination and unchanged register values as the original `breakTo1`. All four tables are included with both dense and native labels. Rooted canonicalization distinguishes them from each other and from the published table.

Physical register permutations offer another useful negative result within a specific allocator. The affected leaf-router actions are transformed by the same injective permutation of skip-chain targets. They cannot serve as neutral bit-preserving discriminators, and equality of the compared actions is preserved. Thus the ordinary three-step context-fold opportunities are invariant under register permutation alone. More general excursions are outside that argument.

No global lower bound. Failure in these finite or heuristic families says nothing about all 195-state machines, or all possible PA-equivalent calculi. The report does not infer impossibility from a search budget, a lack of mutations, or a floating-point optimization bound.

8 Reproduction and remaining work

The ZIP includes the LaTeX source and this compiled PDF, all designated comparison tables and their provenance, the pinned source tree needed by the checkers, the independent mask and difference-bound verifiers, the generalized symbolic executor, search programs and receipts, and the all-instance countervaluation's executable supplement.

The top-level README gives the portable reproduction commands. The combined verification runner checks source/table identities, the exact blank startup, all three inductive certificates, and

the retained symbolic contracts. It does not run the large exploratory searches by default. Those searches have individual commands and frozen outcomes.

To meet the original request, the remaining deliverable is concrete: a table with at most 195 working states, a verified blank startup, sufficient universal instruction or alternative simulation contracts, and a complete PA-formalizable connection to the proof search. None of the new certificates or 196-state variants in this package is a substitute for that missing table.

References

[1] James Chang. *Nonhalting and PA-independence for a 196-state Turing machine*. Paper and sources at revision `407cdc356f6b586635e71ac92be394255f4bfea4`. `https://github.com/jamesyc/BusyBeaver/tree/407cdc356f6b586635e71ac92be394255f4bfea4`.

[2] James Chang. The requested 209-state predecessor, revision `69d98241a721ead20b4f0b17df9062950b40fbfe`. `https://github.com/jamesyc/BusyBeaver/tree/69d98241a721ead20b4f0b17df9062950b40fbfe`.

[3] Vladimir Reshetnikov. ProveIt incoming archives, revision `f1f076e31995f10460838d1846456f4bb9c8ae80`. `https://github.com/VladimirReshetnikov/ProveIt/tree/f1f076e31995f10460838d1846456f4bb9c8ae80/docs/incoming`.

[4] James Chang. *Formal theorem statements*, `docs/result.md`, PA196 revision above. `https://github.com/jamesyc/BusyBeaver/blob/407cdc356f6b586635e71ac92be394255f4bfea4/docs/result.md`.

[5] *PA-consistency machine comparison and reduction research*. Earlier incoming archive `pa_tm_research_2026-11-13.zip`, including comparison inputs, symbolic checker, and reverse-translation argument. Included in the pinned ProveIt incoming directory above.

[6] Vladimir Reshetnikov. ProveIt, *ConciseIPRA*, same pinned revision. `https://github.com/VladimirReshetnikov/ProveIt/tree/f1f076e31995f10460838d1846456f4bb9c8ae80/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`.